Portfolio

Objektorientierte und funktionale Programmierung mit Python
DLBDSOOFPP01_D

IU Internationale Hochschule

Studiengang: Cyber Security

Dashboard für das Studium – Reflexions- und Entwurfsdokument

Marius Burkhardt

Matrikelnummer: IU14107468

Tutor: Aiko Walte

Abgabedatum: 25.10.2026

Inhalt

Abbildungsverzeichnis II

1 Ziel und Vorgehen der Phase 2 1

1.1 Funktionalitäten 1

1.2 Modellierungsentscheidungen/Änderungen Phase 1 1

2 OOP-Konzepte und Python-Umsetzung 1

2.1 Dataclasses 1

2.2 Validierung 1

2.3 Enums 2

2.4 Festgestellte Besonderheiten in Bezug auf Python 2

2.5 Vererbung 2

2.6 Beziehungen 3

2.7 Testprogramme 3

2.8 Mein persönliches Fazit 4

3 Gesamtarchitektur - Schichten und Datenflüsse 5

Abbildungsverzeichnis

Abbildung 1: Tabelle Schichten 5

Abbildung 2: Gesamtübersicht UML 6

1 Ziel und Vorgehen der Phase 2

Das Ziel soll ein Python-Prototyp zur Übersicht des Studienfortschritts im Studiengang Cyber Security sein. Die Architektur orientiert sich am erarbeiteten Phase-1-Modell und den Grundsätzen der Objektorientierten Programmierung, sowie dem generellen Schichtenmodell. Hierbei sollen Verantwortlichkeiten sauber getrennt sein, sowie die Architektur definiert und erklärt werden. Weiterhin wird das UML-Diagramm ergänzt und vervollständigt.

1.1 Funktionalitäten

Zum ersten Start soll eine leere Datenbank angelegt werden. Solange kein Studiengang gespeichert ist, wird in den entsprechenden Feldern die Meldung „Kerndaten anpassen“ gezeigt. Hier soll es zwei Möglichkeiten geben: Kerndaten initial anpassen oder einen gespeicherten Datensatz im JSON-Format einlesen. Ein Datenimport ersetzt die gespeicherten Daten direkt. Eine entsprechende Beispieldatei wird mitgeliefert (examples/beispielstudiengang.json). Das Dashboard soll im oberen Teil die Kerndaten und im linken Teil die Statuswerte enthalten. Funktionale Bedienelemente sollen auf der rechten Seite des Dashboards verfügbar sein. Darin eingebettet soll eine Auflistung der Module ersichtlich sein. Es soll die Möglichkeit geben, Module hinzuzufügen oder zu ändern, die Kerndaten anzupassen und die in der Datenbank gespeicherten Daten, also den kompletten Studiengang, zu importieren oder exportieren.

1.2 Modellierungsentscheidungen/Änderungen Phase 1

Das Phase-1-Klassenmodell beschreibt Fachattribute und Beziehungen. Die Validierung konkreter Eingaben wurde als Umsetzungsentscheidung in Phase 2 ergänzt. Weiterhin wurde für Tests die Bibliothek *pytest* verwendet.

2 OOP-Konzepte und Python-Umsetzung

2.1 Dataclasses

Die Verwendung von @dataclasses vor der Definition der Klassen *Modul, Pruefungsleistung, Semester und Studiengang* nimmt viel manuelle Arbeit ab, da sinnvolle Methoden automatisch erzeugt werden. Somit können mehrere kleine Klassen innerhalb kurzer Zeit mit vergleichbar wenig Code erstellt werden, da Methoden wie beispielsweise __*init*__, oder __eq__ bereitgestellt werden.

2.2 Validierung

Die Notwendigkeit einer Validierung wurde bei ersten Tests schnell bewusst, dass die Basis einer guten Auswertung immer ein gute Datenbasis bedingt. Somit sollte unbedingt geprüft

werden, dass nur gültige Werte eingegeben werden können. Um Benutzerfreundlichkeit zu steigern, sollen entsprechende Fehlermeldungen generiert werden:

```
def __post_init__(self) -> None:
        """Prüft Modulnamen und ECTS-Wert."""
        if not self.name.strip():
            raise ValueError("Der Modulname darf nicht leer sein.")
        if type(self.ects) is not int or self.ects <= 0:
            raise ValueError(
                "Die ECTS-Punktzahl muss eine positive ganze Zahl sein."
            )
```

Die Klasse Module lehnt leere Namen sowie nicht positive oder nicht ganzzahlige ECTS ab.

2.3 Enums

Innerhalb der Klasse *Pruefungsleistung* werden für den *Prüfungstyp* und den *Status* Enums definiert. Somit besteht nur eine fest definierte Anzahl an Auswahlmöglichkeiten, was Schreibfehler und Schreibarbeit minimiert, und eine eventuell später implementierte Filterung erleichtert. Fazit: In diesem Kontext wird bei der Eingabe eine Auswahl erzwungen und diese beschränkt. Folglich werden unvollständige Datensätze vermieden und eine Validierung fällt leichter, da nicht mit Zeichenketten gearbeitet werden muss

2.4 Festgestellte Besonderheiten in Bezug auf Python

- @Dataclass erzeugt nützliche Standardmethoden aber ersetzt nicht die Definition der fachlichen Regeln oder die Validierung von Benutzereingaben eines Modells.
- Typannotationen helfen bei Lesbarkeit und statischer Prüfung, ersetzen aber keine Laufzeitvalidierung.

2.5 Vererbung

An einem hypothetischen Beispiel möchte ich zeigen, warum ich für die Prüfungsarten trotz grundsätzlich möglicher Vererbung keine Unterklassen verwendet habe.

```
from src.domain import Assessment, AssessmentType


class ExamAssessment(Assessment):
    """Experimentelle Prüfungsart mit spezialisiertem Verhalten."""
    def description(self) -> str:
        """Liefert die Beschreibung der Prüfungsart."""
        return "Klausur"


class PortfolioAssessment(Assessment):
    """Experimentelle Prüfungsart mit spezialisiertem Verhalten."""
    def description(self) -> str:
```

```
        """Liefert die Beschreibung der Prüfungsart."""
        return "Portfolio"

def test_assessment_subclasses_inherit_and_specialize_behavior() -> None:
    """Zeigt Vererbung und Methodenüberschreibung für Prüfungsarten."""
    assessments = (
        ExamAssessment(AssessmentType.EXAM),
        PortfolioAssessment(AssessmentType.PORTFOLIO),
    )

    assert all(isinstance(assessment, Assessment) for assessment in assessments)
    assert [assessment.description() for assessment in assessments] == [
        "Klausur",
        "Portfolio",
    ]
```

Das Beispiel zeigt, dass Unterklassen grundsätzlich funktionieren. Allerdings unterscheiden sich die Prüfungsarten nur in ihrer Bezeichnung. Die Eigenschaften Status, Note und Validierung werden jedoch für alle gleich behandelt. Unterklassen wie *Portfolio, Klausur* oder *Workbook* würden an dieser Stelle nur noch mehr Klassen erzeugen ohne eigenes Verhalten abzubilden. Die Prüfungsarten werden deshalb als *Enum* modelliert.

2.6 Beziehungen

StudyProgram enthält eine Liste von Semester-Objekten und jedes Semester enthält eine Liste von Modul-Objekten. Diese Struktur entspricht der Komposition im Fachmodell: Semester und Module sind Bestandteile des Studiengangs und werden in SQLite über Fremdschlüssel gespeichert.

2.7 Testprogramme

Die folgenden pytest-Beispiele prüfen konkrete Modellregeln und machen meine Entwurfsentscheidungen nachvollziehbar:

- Validierung: test_domain.py prüft ungültige Fachwerte; die Konstruktion endet erwartungsgemäß mit ValueError. Code-Ausschnitt:

```
def test_assessment_rejects_invalid_grade_state(
    status: AssessmentStatus, grade: float | None
) -> None:
    """Lehnt nicht passende Status- und Notenkombinationen ab."""
    with pytest.raises(ValueError):
        Assessment(AssessmentType.EXAM, status, grade)
```

- Beziehungen: test_transfer.py importiert verschachtelte Semester-, Modul- und Assessment-Objekte; der Vergleich bestätigt, dass die Struktur erhalten bleibt.

```
@pytest.mark.parametrize(
    ("old_value", "new_value"),
    [
        ('"ects": 5', '"ects": 5.5'),
        ('"ects": 5', '"ects": true'),
        ('"required_ects": 180', '"required_ects": 180.5'),
        ('"required_ects": 180', '"required_ects": true'),
    ],
```

- Enums: test_transfer.py importiert benannte Enum-Werte aus JSON; unbekannte oder fehlende Werte werden als ungültige Eingabe abgelehnt.

2.8 Mein persönliches Fazit

Auf den ersten Blick scheint für den Zweck der Anwendung die Struktur sehr umfangreich. Bei näherer Betrachtung jedoch wird deutlich, dass der Programmcode nicht nur den Zweck der Funktionalität erfüllen sollte, sondern unter anderem auch nach Jahren verständlich, erweiterbar und wartbar sein. Gleichzeitig sind bestimmte Code-Passagen flexibler anpassbar da es den Austausch von kompletten Modulen möglich macht. Durch eine saubere Programmierung wird deutlich mehr Code produziert, als die reine Funktionalität verlangen würde, lässt sich so der Überblick behalten, während Programmteile unabhängig voneinander parallel entwickelt, geplant und getestet werden können. Jedoch ist die Programmierung mit Standardmodulen aufwändiger, was mir am Beispiel mit Tkinter bewusst wurde.

3 Gesamtarchitektur - Schichten und Datenflüsse

Schichtname	Klasse/Modul	Zuständigkeit
Domainklassen	src/domain/	Je eine Datei für Studiengang, Semester, Modul und Prüfungsleistung
Service	dashboard.py	ECTS-, Noten-, Zeit- und Zielberechnungen
Repository	repositories.py storage.py	Laden und Speichern eines Studiengangs; SQLite-Schema und persistente Speicherung
DTO	dto.py	Typisierte, für die Anzeige vorbereitete Daten
Controller	controller.py	Verbindet Repository und Service
Datentransfer	transfer.py	Validierter Import und Export von JSON-Sicherungen
View	ui.py	Darstellung und Benutzereingaben in Tkinter
Application	application.py main.py	Erzeugt und verbindet die Schichten; Programmeinstieg

Abbildung 1: Tabelle Schichten

Das Diagramm zeigt die technische Gesamtarchitektur der Umsetzung. Hierbei wird das sprachunabhängige Modell aus Phase 1 um die Schichten Service, Repository, DTO, Controller, Datentransfer, Oberfläche und Programmeinstieg ergänzt.

In der Application-Schicht befindet sich der Programmeinstieg in main() und ruft die Startfunktion in der application.py auf. Daraufhin werden die verschiedenen Schichten miteinander verbunden und gestartet. Die Application erzeugt das Repository, den DashboardService und den Controller und startet anschließend die View.

Über die Repository-Schicht innerhalb storage.py wird überprüft, ob eine Datenbank bereits existiert. Falls nicht, wird sie initial angelegt mit dem dort definierten Schema. Die konkrete SQLite-Implementierung ist nur der Application-Schicht bekannt.

Innerhalb der Service-Schicht übernimmt der „DashboardService“ die Berechnungen, wenn Daten hinzugefügt oder verändert werden. Die Daten zur Berechnung von Gesamtstatus und Kennzahlen erhält die Service-Schicht aus den Informationen der DTO-Schicht, in der die Daten für die Darstellung aufbereitet werden.

Die View-Schicht übernimmt die optische Darstellung des Dashboards über Tkinter und leitet Benutzeraktionen an den Controller weiter. Dieser lädt und speichert Objekte über das Repository und lässt Kennzahlen über den Service berechnen, und importiert/exportiert die Daten im JSON-Format.

Die DTO-Schicht übernimmt die Datenaufbereitung und stellt die Berechnungen der Service-Schicht für die View bereit. Die Domainklassen werden von Controller, Service und View verwendet, da sie für Formulareingaben benötigt werden.

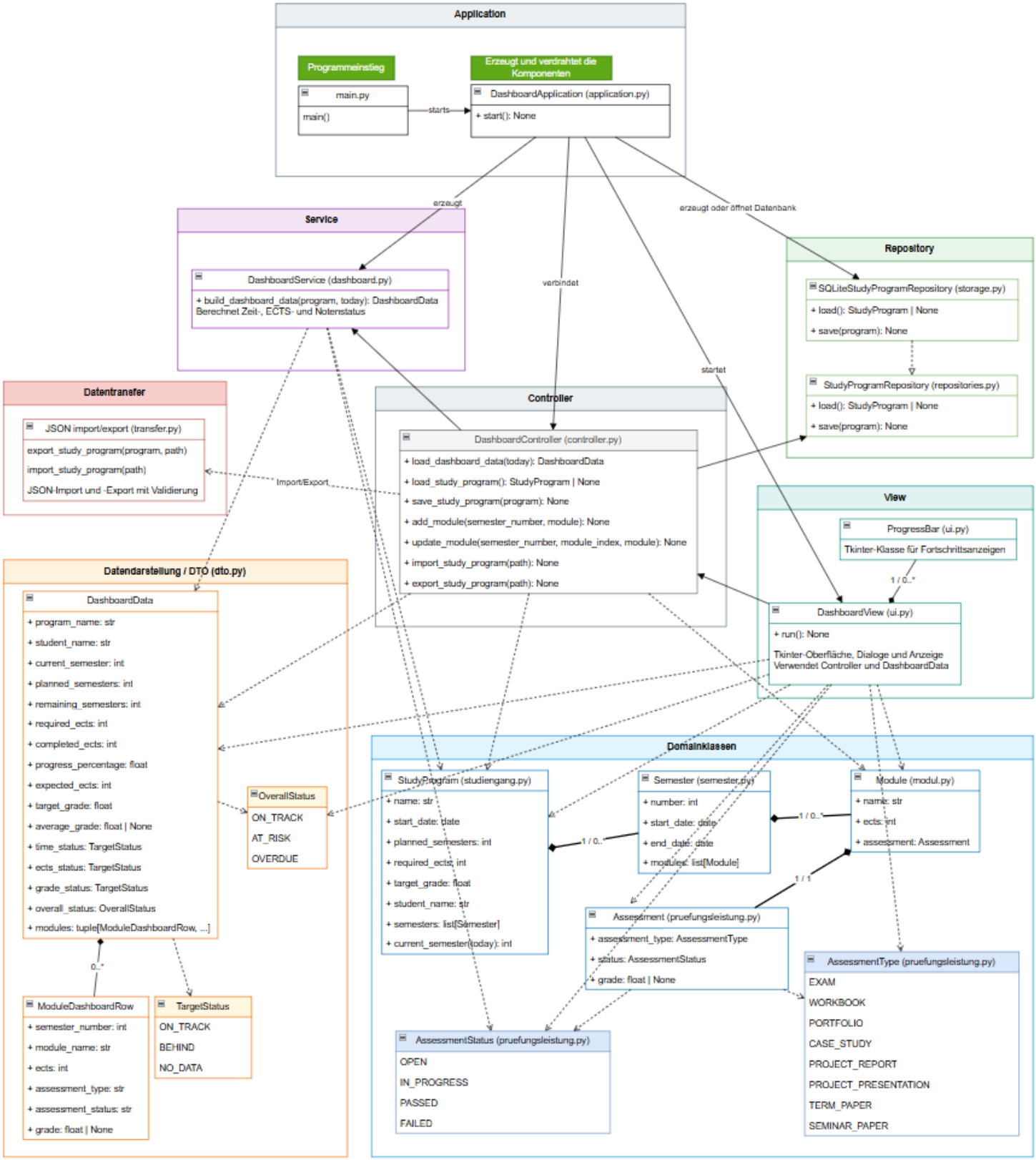


Abbildung 2: Gesamtübersicht UML